# KeyRing: Scalable Join Discovery with LLMs and Recall Guarantees

Anonymous Author(s)

## Abstract

Analytics, BI, and LLM agents over enterprise data first need a join graph: which tables join, on which keys, and in which direction. Each business in an enterprise platform has 50 to 8K tables, rarely declares these keys, and keeps adding tables. Non-LLM methods, such as inclusion-dependency discovery, learned foreign-key detectors, and joinable-table search, are cheap but decide joins from value overlap and names alone. They accept accidental overlaps of surrogate IDs, codes, and dates, and they miss true joins whose values are only partially contained or formatted differently. LLM-based methods add the semantics these methods lack, but current practice sends every pair of tables to the LLM: about 32M calls for 8K tables, and almost all of them return "no join". Each new table then costs one more prompt for every existing table. Hybrid methods cluster tables by topic or route each pair through a cheap proxy before the LLM, but topics are not key domains, proxies inherit the failures of non-LLM signals, and every pair is still judged independently. We present KeyRing, a hybrid system that discovers join keys with LLM calls proportional to the number of *key domains*, not the number of table pairs. KeyRing builds on the observation that enterprise joins concentrate on a few key domains, such as customer or project identifiers, that many tables share. KeyRing uses non-LLM signals for what they do well: it profiles every column once, generates candidates with indexes, and groups candidate key columns into *rings*, one per key domain. It uses the LLM for what only the LLM does well: it verifies the semantics of each ring once, together with a few uncertain candidates. A cost-based optimizer routes each candidate to a local model distilled from earlier LLM decisions or to the LLM, so that KeyRing keeps a target recall of the all-pairs LLM baseline with a user-given confidence. Joins follow from ring membership, so the resulting graph is a star around each key owner instead of a clique of redundant edges. KeyRing runs online: when a table arrives, it looks up the new columns in its indexes and verifies only the rings they affect, so the cost of an arrival does not grow with the number of tables. On [N] enterprise databases with up to [8K] tables and on public benchmarks, KeyRing improves join F1 over the best non-LLM method by [P] points, and reduces LLM calls by [X]× and tokens by [Y]× compared to all-pairs prompting, while keeping [Z]% of its joins. When tables arrive one at a time, KeyRing adds each table with [V]× fewer LLM calls than pairwise prompting and returns the same graph as a batch run.

**ACM Reference Format:**
Anonymous Author(s). 2028. KeyRing: Scalable Join Discovery with LLMs and Recall Guarantees. In *Proceedings of SIGMOD (SIGMOD'28).* ACM, New York, NY, USA, 9 pages. https://doi.org/XXXXXXX.XXXXXXX

*SIGMOD'28, TBD*
© 2028 Copyright held by the owner/author(s).
ACM ISBN 978-1-4503-XXXX-X/2018/06
https://doi.org/XXXXXXX.XXXXXXX

## 1 Introduction

Enterprise data platforms ingest the databases of many business units, ERP systems, and exported files. Before analysts, BI tools, or LLM agents can query this data, the platform needs a *join graph*: which tables join, on which key columns, and in which direction. Text-to-SQL over large schemas [17] and automatic BI modeling [18] both depend on it. However, enterprise databases rarely declare foreign keys. Tables arrive as staging copies, snapshots, and exports, and one business can have 50 to 8K tables. The join graph must therefore be discovered from the data and its metadata. It is also never finished: the platform onboards new source systems and exports, so tables keep arriving, and each new table must be linked to the graph before it can be queried. We discover joins within one business; joins across businesses are out of scope.

**Example.** Consider a project-based services firm with [2,010] tables. A *project code* identifies a project in the project master table, and [k] other tables reference it: ledgers, timesheets, invoices, budgets, and their staging copies and snapshots. In some tables the column is called `PROJECT_CODE`, in others `PRJ_ID` or `ENGAGEMENT`. Staging copies keep only recent projects, and one export pads codes with leading zeros. At the same time, dozens of tables have auto-increment surrogate columns that start at 1, status columns with values 1 to 5, and a generic column named `DATA`. A correct join graph links the [k] referencing columns to the project master, and nothing else.

**Non-LLM Join Discovery.** The classic approach decides joins from data statistics. Inclusion-dependency algorithms find columns whose values are contained in a unique column [32]. Learned detectors score such candidates with features such as uniqueness, containment, and name similarity [10, 29, 36], and Auto-BI [18] combines a local join classifier with a global graph optimization. Joinable-table search treats each column as a *domain*, the set of its distinct values, and finds the columns whose domains overlap most with a query column. LSH Ensemble [38] showed that set containment $|Q \cap X|/|Q|$, not Jaccard similarity, is the right overlap measure when domain sizes are skewed, and indexes it with MinHash sketches at the scale of hundreds of millions of domains. JOSIE [37], MATE [8], DeepJoin [7], and Freyja [21] extend this search to exact overlap, multi-column keys, embeddings, and learned join quality. These methods are cheap and scale to large data lakes. However, on the example above they fail in four ways.

- **Accidental overlap creates false joins.** Every surrogate column with values $1..m$ is contained in every other surrogate column with values $1..n$, $n \geq m$. Status codes, years, and flags overlap in the same way. Value statistics cannot distinguish these overlaps from real references, so the graph fills with false edges. LakeBench [5] reports the same failure for joinable-table search: JOSIE returns a column of street names for a query column of last

names, because both contain values such as "Lincoln" and "Washington". In our databases, [P]% of the inclusion dependencies with containment above 0.9 are not joins.

- **Real joins look imperfect.** Staging copies, deleted rows, and snapshots make containment partial, and formatting differences, such as leading zeros or prefixes, make it zero. A strict containment threshold misses these joins, and a loose one admits more accidental overlaps. No single threshold works across a database.
- **Names do not carry meaning.** The same key appears under different names (`PRJ_ID`, `ENGAGEMENT`), and different keys share generic names (`ID`, `CODE`, `DATA`). String similarity on names cannot resolve either case. The column descriptions and business context that would resolve them are text that these methods do not read.
- **Learned models do not transfer.** Learned detectors and Auto-BI need labeled joins from schemas similar to the target. Every business has its own naming conventions and source systems, and labels are expensive: LakeBench [5] spent more than 7,500 human hours to label its join and union queries. Joinable-table search returns a ranked list per query column, not a decision with a key direction.

**LLM-based Join Discovery.** LLMs address these failures because they read column names, descriptions, and sample values together, and they bring knowledge of common business entities [23]. An LLM can tell that `ENGAGEMENT` holds project codes and that a surrogate `ID` does not reference a status table. Current practice therefore sends every pair of tables (source and target) to an LLM, which decides whether the pair joins and returns the key. This approach is accurate, but its cost grows quadratically with the number of tables. A business with $n$ tables needs $n(n-1)/2$ pair evaluations: about 500K for 1K tables and about 32M for 8K tables. Each evaluation is itself several LLM calls that carry schemas, descriptions, and sample values, so a single business costs [$X] in tokens and [Y] hours of inference. The cost does not stop after the first build: every new table is evaluated against all $n$ existing tables, and nothing learned from earlier pairs is reused. The calls are also mostly wasted: only [Z]% of the table pairs in our databases join. In the example, the LLM is asked about each of the $\binom{k+1}{2}$ pairs among the project tables, although the answers follow from the $k$ references to the project master. Independent answers can also contradict each other, and they add a clique of redundant edges to the join graph. Redundant edges are not harmless. Downstream tools that pick one join path per table pair choose among the equivalent paths arbitrarily, so a new shortcut edge can silently change which tables and keys a query uses, and every extra edge is paid for once when it is discovered and again whenever the graph is placed in a prompt ([**measure on production**]).

**Hybrid Join Discovery.** A natural fix is to combine cheap methods with the LLM. Hybrids do so in two ways.

The first way *narrows the search space* before the LLM. Schuyler [16] clusters the tables of a database into topics with self-supervised LLM embeddings, and notes that such clusters reduce the search for joinable tables to matching within and across groups. Our production pipeline follows the same idea: an LLM tags each table and groups related tables into concepts before it searches for joins. Magneto [20] uses a small model to retrieve candidate column matches for an LLM. However, topics are not key domains. A customer or project key crosses topic boundaries, so restricting the search to a cluster loses joins, and searching across clusters brings back the quadratic number of pairs.

**Table 1: Join key discovery over thousands of tables: non-LLM methods are cheap but decide from overlap and names, all-pairs LLM prompting calls the LLM per table pair, hybrids still judge pairs independently; KeyRing calls the LLM per key domain and keeps a target recall.**

| | Non-LLM | | LLM | Hybrid | Hybrid |
|---|---|---|---|---|---|
| | **Auto-BI** [18] | **JOSIE** [37] | **All-pairs** prompting | **LOTUS** [28], **FDJ** [34] | **KeyRing** ours |
| **Robust to accidental overlap** | ✗ learned syntactic features | ✗ value overlap | ✔ | ✔ if routed to LLM | ✔ LLM checks each ring |
| **Finds joins with partial or zero containment** | ✗ | ✗ | ✔ | ✗ if proxy drops them | ✔ normalized formats, embeddings |
| **Labeled data needed** | ✗ corpus of BI models | ✔ none | ✔ none | ✔ none | ✔ own LLM decisions |
| **LLM calls grow with** | n/a | n/a | ✗ table pairs | ✗ candidate pairs | ✔ key domains |
| **Recall guarantee** | ✗ | ✗ top-$k$ by overlap | n/a (asks all pairs) | ✔ per operator | ✔ vs. all-pairs LLM |
| **Returns key and direction** | ✔ | ✗ | ✔ | ✗ user predicate | ✔ owner and members |
| **Consistent, non-redundant graph** | ✔ arborescence | ✗ | ✗ independent answers | ✗ | ✔ star per key domain |
| **Cost to link a new table** | ✗ re-solve global graph | ✔ index lookup | ✗ $n$ pair prompts | ✗ $n$ pair operators | ✔ affected rings only |
| **F1; cost** ([N] tables) | [F1]; no LLM | [F1]; no LLM | [F1]; [$, h] | [F1]; [$, h] | [F1]; [$, h] |

The second way *routes each pair* to a cheap proxy or to the LLM. Semantic join operators in LOTUS [28] and Palimpzest [19], and model cascades [4], accept or reject a pair by its embedding similarity and defer inconclusive pairs to the LLM. SUPG [11] and BARGAIN [35] set the deferral thresholds so that the result keeps a target precision or recall of the LLM with statistical guarantees. Such cascades save little when embeddings do not encode the join condition: on real data, BARGAIN still defers about 80% of pairs to the LLM [34]. Featurized-Decomposition Join (FDJ) [34] reduces the cost by up to 10× with a key idea: extract features from each record with a linear pass of the LLM, and decide the join with cheap comparisons on those features.

These hybrids lower the cost per pair, but they inherit two problems. First, a cheap filter or proxy inherits the failures of non-LLM methods: a surrogate overlap passes it and costs an LLM call, and a real join with zero containment is dropped before the LLM sees it. Second, they evaluate a given join predicate over record pairs of two given tables. Join discovery asks a different question: which of thousands of tables join, and on which of their columns. Applied to table or column pairs, these operators judge every pair independently. They do not use what makes join keys special: one side is unique, the other side references it, and many tables share one key domain. The number of LLM calls still grows with the number of candidate pairs.

**Problem and Challenges.** Given a database without declared keys and an LLM, we aim to discover its join keys with as few LLM

calls as possible, while keeping a target recall of the all-pairs LLM baseline. Table 1 compares one representative per category against this goal. Because tables keep arriving, the system must also link each new table at a cost that does not grow with the size of the business. Meeting these goals is hard for four reasons. First, the candidate space is quadratic and almost all of it is empty, so pruning must remove nearly all pairs before any LLM call, yet every true join it drops is lost silently. Second, the cheap signals that pruning relies on are exactly the ones that fail in non-LLM methods. Third, pairwise decisions ignore the structure of key domains, so they are redundant and can be inconsistent. Fourth, an online system sees the tables in an arbitrary order. A table that arrives late may be the true owner of a key that many earlier tables reference, so earlier decisions must be revised, and the result should not depend on the order of arrival.

**Our Insight.** Enterprise joins are not spread evenly over table pairs. They concentrate on a small number of *key domains*, such as customer, project, employee, or invoice identifiers. Each key domain has one *owner* column that is unique, and many *member* columns that reference it. If we know the key domains, we know the joins: every member joins with its owner. The number of key domains is orders of magnitude smaller than the number of table pairs. So the LLM should verify key domains, not pairs. This lifts FDJ's insight from records to schemas: work that is linear in the number of columns (profiling, annotation, and grouping columns into key domains) replaces work that is quadratic in the number of tables. It also extends the value domains of LSH Ensemble with meaning and ownership: a key domain is a set of columns that refer to the same business entity, with one owner. Finally, it splits the work between the two kinds of methods along their strengths. Non-LLM signals are reliable for uniqueness and for proposing candidates with high recall, but not for deciding meaning. The LLM is reliable for meaning, but too expensive to apply to every pair.

**System Overview.** This paper presents KeyRing[1], a hybrid system that discovers join keys with LLM calls proportional to the number of key domains. KeyRing has three core components. First, the *Column Profiler* reads the data once and computes sketches of each column's values, its uniqueness, type, value pattern, and normalized formats, and an embedding of its name and description. No later step reads raw data. Second, the *Ring Builder* generates candidates with containment and embedding indexes, tuned for recall rather than precision. It prunes generic and type-incompatible columns, and groups the remaining columns into *rings*, one per candidate key domain, each with an elected owner. Third, the *Budgeted Verifier* decides which rings and candidates need the LLM. A local model, distilled from KeyRing's own earlier LLM decisions rather than from labeled schemas, accepts or rejects confident candidates. The LLM verifies each ring once, in a batched prompt with its owner, its members, and its uncertain candidates, and removes accidental members such as surrogate overlaps. A cost-based optimizer chooses thresholds and batches so that KeyRing keeps a target recall $\gamma$ of the all-pairs LLM baseline with probability at least $1 - \delta$. Confirmed rings propagate joins by transitivity without further LLM calls. The output join graph is a star around each owner, not a clique of redundant edges.

KeyRing runs in two modes over the same state: profiles, indexes, rings, and the distilled model. In *bootstrap* mode, it builds the rings of a business from all its tables. In *online* mode, it links one arriving table: it profiles only the new columns, looks them up in the existing indexes, and assigns each key-like column to a ring or starts a new one. A fourth component, the *Ring Maintainer*, revises the affected rings when an arrival changes them: it elects a better owner, merges two rings that the new table connects, or splits a ring that verification shows to mix two domains. The Budgeted Verifier then checks only the affected rings. Each change updates the edges of one ring, so the cost of an arrival grows with the size of the new table and of the rings it touches, not with the number of tables. Bootstrap is the special case in which all tables arrive at once, and we show that the final rings do not depend on the order of arrival.

**Contributions.** Our primary contribution is KeyRing, a holistic and practical hybrid system for join key discovery over thousands of tables that is more accurate than non-LLM methods and whose LLM cost grows with the number of key domains, not with the number of table pairs. Our detailed technical contributions are:

- An analysis of why non-LLM, LLM-based, and hybrid join discovery fail on enterprise databases, a formalization of cost-bounded join key discovery with a recall target relative to an LLM oracle, and a system architecture that solves it by discovering key domains instead of judging table pairs (Sections 2 and 3).
- A ring builder that turns one pass of column profiling into key-domain candidates with elected owners, using containment and embedding indexes, format normalization, and pruning of generic columns (Section 4).
- A budgeted verifier that combines a distilled local model, batched LLM prompts per ring, and transitive propagation, and a cost-based optimizer that meets a recall target with statistical guarantees (Section 5).
- An online mode that links each arriving table by index lookups and verification of the affected rings only, with a ring maintainer that re-elects owners and merges or splits rings, so that the cost of an arrival is independent of the number of tables and the result is independent of the order of arrival (Section 6).
- A comprehensive evaluation on [N] production enterprise databases with up to [8K] tables and on public benchmarks, against non-LLM methods (inclusion-dependency discovery, Auto-BI, joinable-table search), all-pairs LLM prompting, and hybrid methods (Section 7). KeyRing improves join F1 over the best non-LLM method by [P] points, and reduces LLM calls by [X]×, tokens by [Y]×, and end-to-end time by [W]× compared to all-pairs prompting, while keeping [Z]% of its joins. Replaying the tables of each business in their creation order, KeyRing links each arriving table with [V]× fewer LLM calls than pairwise prompting and ends with the same graph as a bootstrap run.

## 2 Background and Problem Formulation

In this section, we review the three categories of join discovery from Table 1: non-LLM, LLM-based, and hybrid. For each category, we describe how it decides joins and where it falls short on enterprise

[1] KeyRing holds all columns of one key domain together, like keys on a ring. The code is available to reviewers at https://anonymous.4open.science/r/KeyRing.

databases. We then review how profiling metadata is collected and provided to LLMs, and formalize cost-bounded join key discovery.

## 2.1 Non-LLM Join Discovery

Non-LLM methods decide joins from data statistics and schema metadata. They fall into two groups: key and inclusion-dependency discovery, which decides joins inside one database, and joinable-table search, which ranks candidate columns for a query column in a data lake.

**Key and Inclusion-Dependency Discovery.** A unary *inclusion dependency* (IND) $A \subseteq B$ holds if every value of column $A$ also appears in column $B$. If $B$ is also unique, then $A$ is a foreign-key candidate that references $B$. IND discovery algorithms check this condition for all column pairs, and scale to databases with very many tables by sorting and sharing value sets across checks [32]. Real data rarely satisfies exact INDs, so approximate INDs relax the condition to a containment threshold. Every IND is only a candidate, and most candidates are not foreign keys. Learned detectors therefore rank candidates with features, such as the uniqueness of $B$, the containment of $A$ in $B$, the similarity of column names, and the distribution of values [29, 36]. HoPF [10] detects primary and foreign keys together, so that a column chosen as a primary key constrains which foreign keys may reference it. Auto-BI [18] goes one step further. It first trains a local classifier that predicts, for each column pair, whether it is an N:1 join. It then selects a global join graph that maximizes the combined classifier scores under structural constraints, a variant of the minimum-cost arborescence problem. The global step removes many locally plausible but inconsistent joins.

**Joinable-Table Search.** Data lake systems treat each column as a *domain*, the set of its distinct values, and rank the columns of a lake by their overlap with a query column. LSH Ensemble [38] uses set containment $|Q \cap X|/|Q|$ as the overlap measure. Unlike Jaccard similarity, containment does not penalize a large domain $X$ for values outside $Q$, which matters because domain sizes follow a power-law distribution. LSH Ensemble indexes MinHash sketches of all domains in partitions of similar size, and answers containment queries over hundreds of millions of domains in seconds. JOSIE [37] returns the exact top-$k$ columns by set overlap and chooses between reading posting lists and reading candidate sets with a cost model. MATE [8] extends the search to multi-column keys. DeepJoin [7] embeds columns with a pre-trained language model and searches them with an approximate nearest-neighbor index, which also finds joins with similar but not identical values. Freyja [21] learns a join quality metric from column profiles instead of raw values.

**Limitations.** Non-LLM methods are cheap, but they decide joins from syntactic evidence, and enterprise databases defeat this evidence in four ways. First, accidental overlap creates false joins. Surrogate keys that count from 1, status codes, years, and flags are contained in each other, and a unique surrogate column looks like a valid target for all of them. LakeBench [5] shows the same problem for joinable-table search: a column of street names overlaps a query column of last names, and JOSIE ranks it among the top results although the two columns are not joinable. Second, real joins look imperfect. Staging tables, snapshots, and deleted rows make containment partial, and formatting differences make exact overlap zero. On LakeBench queries whose joins have only fuzzy overlap, JOSIE reaches 0% top-10 accuracy and DeepJoin reaches 33% [5]. LakeBench concludes that no method is effective in all cases, because joinable columns must both mean the same thing and overlap in value [5]. Third, names do not resolve meaning. One key appears under different names, and different keys share generic names such as `ID`, `CODE`, or `DATA`. Column descriptions and business context would resolve these cases, but these methods do not read them. Fourth, learned models need labels. The detectors and Auto-BI learn from labeled joins, and labels for a new enterprise schema are expensive: LakeBench spent more than 7,500 human hours to label its queries [5]. Joinable-table search, in turn, returns a ranked list per query column, not a join decision with a key direction.

## 2.2 LLM-based Join Discovery

LLMs decide what columns mean. Foundation models match entities and schema elements across tables without task-specific training [23], and they read names, descriptions, and sample values together. This resolves the cases that defeat non-LLM methods: an LLM recognizes that `ENGAGEMENT` holds project codes, that a surrogate `ID` does not reference a status table, and that a street name is not a last name.

**All-Pairs Prompting.** Current practice in enterprise data platforms, including the production pipeline we study, discovers joins in three LLM steps. First, the LLM annotates every table: it tags its columns with semantic types and writes a description. Second, it tags each table with the *concepts* of a business data model, such as project or invoice, whose data the table holds. Third, it discovers joins pair by pair. For each pair of tables, one prompt removes the columns that cannot be keys, a second prompt selects the candidate key columns of one table, and one further prompt per candidate lists all columns of the other table with their descriptions and sample values and returns the matching columns. Warehouse queries then score the matches by value overlap and uniqueness and assemble a composite key greedily. The first two steps make one LLM call per table or per batch of tables, so their cost is linear in the number of tables. The third step dominates the cost.

**Limitations.** All-pairs prompting is accurate, but it faces four limitations. First, its cost grows quadratically. A database with $n$ tables needs $n(n-1)/2$ pair evaluations, about 32M for 8K tables, and each evaluation makes $2+c$ LLM calls for $c$ candidate columns, most of which carry a full schema and sample values. In our databases, only [Z]% of the pairs join, so almost all calls return "no join". Second, its calls are redundant. If $k$ tables reference one key, the LLM answers $O(k^2)$ questions whose answers follow from $k$ references to the owner of the key. Third, its answers are independent, so nothing makes them consistent. The LLM may confirm that $A$ joins $B$ and $B$ joins $C$ on the same key but reject that $A$ joins $C$. It also cannot see global evidence that is obvious across pairs: in one of our databases, it linked a generic column named `DATA` to [1,826] other columns, one pair at a time. The resulting join graph contains cliques of redundant edges and false edges that are hard to remove afterwards. Fourth, it is incremental only in the trivial sense. It skips pairs it has already evaluated, so a new table costs $n$ new pair evaluations, and every evaluation re-derives facts that earlier pairs already established, such as which column of a table holds customer identifiers.

## 2.3 Hybrid Join Discovery

Hybrid methods combine cheap methods with the LLM to cut the number of LLM calls. They either narrow the search space before the LLM, or route each candidate to a cheap proxy or to the LLM.

**Narrowing the Search Space.** Schuyler [16] clusters the tables of a relational database into topics. It fine-tunes a language model with a triplet loss on self-supervised signals from the schema, embeds each table, and clusters the embeddings, without labeled data. Its authors note that such clusters reduce the search for joinable and unionable tables, because integration can match groups before matching tables. The concept step of all-pairs prompting follows the same idea. Magneto [20] narrows schema matching between two tables: a small language model retrieves candidate column matches, and an LLM reranks them.

**Routing Candidates.** LLM-powered data processing systems, such as LOTUS [28] and Palimpzest [19], provide a *semantic join*: a join of two tables whose condition is a natural-language predicate evaluated by an LLM on each pair of records. To avoid evaluating the full cross product, they use a *model cascade* [4]. A cheap proxy, typically the cosine similarity of record embeddings, scores each pair. Pairs with a low score are rejected, pairs with a high score are accepted, and only pairs in between go to the LLM. SUPG [11] and BARGAIN [35] set the two thresholds from a sample of LLM labels, so that the output reaches a target precision or recall of the all-LLM result with probability at least $1 - \delta$. The savings depend on the proxy. When embeddings do not encode the join condition, most pairs fall between the thresholds: on real-world documents, BARGAIN still defers about 80% of pairs to the LLM [34]. Featurized-Decomposition Join (FDJ) [34] replaces the single proxy with a *featurized decomposition*. An LLM extracts features from each record in one linear pass, such as dates, locations, and names. A logical expression in conjunctive normal form over cheap comparisons of these features then prunes non-matching pairs, with thresholds set to keep statistical guarantees. FDJ reduces cost by up to 10× over BARGAIN [34].

**Limitations.** Hybrids reduce the cost per pair, but they fall short for join discovery in two ways. First, their cheap stage inherits the failures of non-LLM methods. Topical clusters are not key domains: a customer or project key is referenced from every topic, so a search restricted to clusters loses joins, and a search across clusters returns to a quadratic number of pairs. A proxy based on overlap or embeddings passes accidental overlaps to the LLM and drops joins whose values do not overlap before the LLM sees them. Second, semantic join operators answer a different question. They evaluate a *given* predicate over the records of *two given* tables. Join discovery must find which of thousands of tables join and on which columns. Applied to table or column pairs, a semantic join still judges every pair independently. It does not use the structure of join keys: the referenced side is unique, the referencing side is contained in it, and many tables share one key. FDJ shows that linear per-record extraction can replace quadratic per-pair decisions. KeyRing applies this principle to schemas, with key domains as the extracted structure.

## 2.4 Profiling Metadata for LLMs

All methods above consume column profiles, and LLM-based methods must also fit a description of each column into a prompt. We review how dependencies are discovered and stored, how LLMs annotate columns, and how metadata is selected for a prompt.

**Dependency Discovery and Profile Stores.** Data profiling computes metadata such as unique column combinations (UCCs), functional dependencies (FDs), and INDs [1]. Metanome [24] bundles discovery algorithms for these dependencies. HyFD [26] and HyUCC [27] combine approximation on samples with validation to find all minimal FDs and UCCs, and BINDER [25] finds all INDs by partitioning the value sets of all columns. The number of candidate dependencies grows exponentially with the number of columns, so exact discovery over a full warehouse is expensive. Approximate algorithms reduce this cost. FAIDA [15] tests IND candidates on HyperLogLog sketches and samples, and reports no false positives in its evaluation; Pyro [14] discovers approximate FDs and UCCs that tolerate errors in the data. DynFD [30] maintains FDs as the data changes, instead of discovering them again. Metacrate [13] stores them as a queryable database of data profiles. Wan and Han [33] argue that a valid dependency is not always a useful one: discovery should target the dependencies whose value for a downstream task exceeds the cost to discover and maintain them.

**LLM-based Profiling and Annotation.** LLMs annotate columns with semantic types without task-specific training [9, 12], and write column descriptions for catalogs and downstream tasks [3]. Dependencies can also come from the LLM, not the data: LLMClean [2] lets an LLM derive ontological FDs from a table and uses them to detect errors. DBAutoDoc [22] documents databases without declared keys. It combines statistical analysis with iterative LLM refinement, which propagates corrections along the schema's dependency graph, and its statistical stage improves foreign-key detection by 23 F1 points over an LLM-only approach.

**Selecting Metadata for LLM Prompts.** A profile helps an LLM only if its relevant part fits into the prompt. Shkapenyuk et al. [31] summarize database profiles with an LLM and select schema elements for text-to-SQL with a recall-oriented schema linking algorithm. On BIRD with GPT-4o, profile-based metadata raises execution accuracy from 49.8% to 61.2%, and schema linking raises it to 63.2% [31]. REVEAL [6] annotates a column with a compact subset of the other columns of its table, selected for relevance and diversity, instead of the whole table, and improves column type and column property annotation.

**Limitations.** These works either collect dependencies or select context, but not both for one task. Profiling systems discover all valid dependencies of a type and leave their use to the application. Context selection methods choose columns or schema elements, not dependencies. To our knowledge, no method decides which dependencies to discover, and which to show the LLM, for a given column and task under a token budget. Join discovery needs exactly this decision. An IND into a unique column suggests a reference, the description of the referenced key tells the LLM what the referencing column means, and most other dependencies do not affect the decision. KeyRing makes this selection with key domains: a ring prompt contains the owner of a key domain and its candidate members, together with the profile evidence that connects them.

## 2.5 Problem Formulation

Let $\mathcal{D}$ be the database of one business, with tables $\mathcal{T} = \{T_1, \ldots, T_n\}$, and let $C$ be the set of all their columns. A business typically has 50 to 8K tables. Businesses are independent: every definition below is relative to one $\mathcal{D}$, and joins across businesses are out of scope. Each column $c \in C$ has a name, an optional LLM annotation (semantic type and description), and a set of values $V(c)$. We write $u(c) = |\text{distinct}(V(c))|/|V(c)|$ for its uniqueness, and $\text{cont}(a, b) = |V(a) \cap V(b)|/|V(a)|$ for the containment of $a$ in $b$.

*Definition 2.1 (Join Key).* A *join key* $a \to b$ is an ordered pair of columns $a, b \in C$ of different tables, where $b$ identifies a business entity and $a$ refers to the same entity. We call $b$ the *referenced* column and $a$ the *referencing* column.

The definition is semantic on purpose. Uniqueness and containment are evidence for a join key, but neither is necessary nor sufficient on enterprise data, as Section 2.1 shows. We therefore define the ground truth by an oracle.

*Definition 2.2 (LLM Oracle).* An *oracle* $O$ is an LLM with a fixed prompt that, given two tables, returns the set of join keys between them. The oracle join set $J^* = \bigcup_{i<j} O(T_i, T_j)$ is the result of all-pairs prompting. Computing $J^*$ costs $n(n-1)/2$ oracle calls.

Using the oracle as the reference makes the goal precise: a cheaper method should return what all-pairs prompting would return, at a fraction of its cost. It also matches how such systems are evaluated in practice, where the all-LLM result is the reference for cascades [11, 34, 35]. We additionally report precision and recall against human-labeled joins on a sample (Section 7).

*Definition 2.3 (Key Domain and Ring).* A *key domain* is a business entity that columns refer to, such as a customer or a project. A *ring* $R = (o_R, M_R)$ represents one key domain by an *owner* column $o_R \in C$ that identifies the entity, and a set of *member* columns $M_R \subseteq C \setminus \{o_R\}$ that refer to it. A ring induces the join keys $J(R) = \{m \to o_R \mid m \in M_R\}$. A set of rings $\mathcal{R}$ induces the join graph $G(\mathcal{R})$ with edges $\bigcup_{R \in \mathcal{R}} J(R)$.

Two members of the same ring also join with each other, through their shared owner. $G(\mathcal{R})$ does not store these joins as edges, because they follow by transitivity. So a key referenced by $k$ columns produces $k$ edges in $G(\mathcal{R})$, a star around its owner, instead of the $O(k^2)$ edges of a clique.

**Cost Model.** Let $\text{cost}(P)$ be the cost of an LLM prompt $P$, measured in tokens and converted to dollars by the price of the model. A method $\mathcal{A}$ issues a sequence of prompts $P_1, \ldots, P_m$ and returns a join set $\hat{J}$. Its LLM cost is $\text{cost}(\mathcal{A}) = \sum_{i=1}^{m} \text{cost}(P_i)$. We also report its number of LLM calls $m$ and its end-to-end time, including profiling and all non-LLM steps. The recall of $\hat{J}$ with respect to the oracle is $\text{rec}(\hat{J}) = |\hat{J} \cap J^*|/|J^*|$, where a member-to-member join implied by a ring counts as found.

**Optimization Objective.** Given $\mathcal{D}$, an oracle $O$, a recall target $\gamma \in (0, 1]$, and a confidence level $1 - \delta$, find a set of rings $\mathcal{R}$, and thereby a join set $\hat{J} = G(\mathcal{R})$, that minimizes the LLM cost:

$$\min_{\mathcal{A}} \ \text{cost}(\mathcal{A}) \quad \text{s.t.} \quad \Pr\left[\text{rec}(\hat{J}) \geq \gamma\right] \geq 1 - \delta, \tag{1}$$

subject to five requirements.

- *(R1) Recall-guaranteed:* $\hat{J}$ keeps at least a fraction $\gamma$ of the oracle joins with probability at least $1 - \delta$, where the probability is over the random samples the method draws.
- *(R2) Domain-proportional:* the number of LLM calls grows with the number of key domains and uncertain candidates, not with the number of table pairs.
- *(R3) Label-free:* the method needs no labeled joins and no corpus of similar schemas. It may learn only from the oracle answers it pays for.
- *(R4) Consistent:* $\hat{J}$ is induced by rings, so every referencing column points to the owner of its key domain, and joins implied by transitivity are not stored as separate edges.
- *(R5) Online:* when a table arrives, the method updates $\hat{J}$ at a cost that grows with the size of the new table and of the rings it affects, not with $n$, and the final $\hat{J}$ does not depend on the order in which tables arrive.

A precision constraint can be added in the same form; we report precision for all methods.

**Online Setting.** Tables arrive over time. Let $T_1, T_2, \ldots$ be the order of arrival, $\mathcal{D}_t = \{T_1, \ldots, T_t\}$ the business after $t$ arrivals, and $J_t^*$ the oracle join set of $\mathcal{D}_t$. On the arrival of $T_{t+1}$, a method receives its columns and their profiles and annotations, and returns the updated join set $\hat{J}_{t+1}$. It may add edges incident to $T_{t+1}$, and it may also revise earlier edges, for example when $T_{t+1}$ holds a better owner for an existing ring. Requirement R1 must hold after every arrival, with $J_t^*$ as the reference, and the objective is the total LLM cost over the whole sequence. The batch problem above is the special case of a single arrival that contains all tables.

In Table 1, non-LLM methods meet R2 and R4 at best but have no oracle-level semantics, and Auto-BI violates R3. All-pairs prompting is the oracle itself and violates R2, R4, and R5, because each arrival costs $n$ new pair evaluations. Hybrid semantic joins meet R1 and R3 for a given pair of tables, but applied to join discovery they violate R2, R4, and R5. Joinable-table search meets the cost part of R5, because a new column is a single index lookup, but it returns ranked candidates rather than joins.

## 3 The KeyRing System Architecture

KeyRing has three core components that run as a pipeline, and a fourth, the Ring Maintainer, that updates their result when a table arrives. First, the *Column Profiler* reads the data once and summarizes every column. Second, the *Ring Builder* turns these summaries into candidate key domains, called rings, without any LLM call. Third, the *Budgeted Verifier* decides which candidates need the LLM, verifies each ring with one batched prompt, and returns the join graph. Only the third component calls the LLM. Figure 1 shows the three core components along the top, the Ring Maintainer and the path of an arriving table along the bottom, and the data that flows between them. Grey cards are components that the production pipeline already has; KeyRing reuses them.

**Two Modes.** The components keep their state between runs: the profiles, the two candidate indexes, the rings with their owners, the distilled model and its thresholds, and the join graph. KeyRing uses this state in two modes. In *bootstrap* mode, it runs the pipeline over all tables of a business, when the business is onboarded or rebuilt. In *online* mode, it links one arriving table. The profiler

summarizes only the new columns, the Ring Builder looks them up in the existing indexes instead of rebuilding the rings, a fourth component, the *Ring Maintainer*, applies the resulting changes to the affected rings, and the Budgeted Verifier checks only those rings. Both modes share the same candidate generation, scoring, and verification, so a ring confirmed in one mode is valid in the other.

**Design Principles.** Three principles shape KeyRing, each serving requirements from Section 2.5. First, KeyRing decides key domains, not table pairs. The LLM verifies a ring once, and the joins of all its members follow from that answer, so the number of LLM calls grows with the number of key domains (R2), and the output is a star around each owner (R4). Second, KeyRing uses each signal for what it is reliable at. Uniqueness, containment, and embeddings are reliable for proposing candidates, but not for deciding what a column means. The Ring Builder therefore tunes them for recall, because a join it drops cannot be recovered later, and leaves meaning to the LLM. Third, KeyRing pays for the LLM only where its answer changes the outcome, and it reuses every answer it pays for. A local model, trained only on KeyRing's own LLM answers, decides confident candidates (R3), and a cost-based optimizer sets its thresholds so that the result meets the recall target (R1).

## 3.1 Column Profiler

The Column Profiler makes one pass over the data and computes a profile for each column $c \in C$. The profile has four parts, shown on the left of Figure 1. First, *value sketches*: a MinHash sketch of the distinct values for containment estimates [38], and a HyperLogLog sketch for the number of distinct values. Second, *statistics*: the uniqueness $u(c)$, the null ratio, the data type, and a value pattern, such as "three letters, a dash, and five digits". Third, *normalized formats*: the profiler strips leading zeros, prefixes, and case differences, and sketches the normalized values as well, so that a join with zero exact overlap can still show containment. Fourth, an *embedding* of the column name together with its LLM annotation, if the annotation step of the current pipeline (Section 2.2) has already produced one. The statistics and the LLM annotation already exist in the production pipeline; the sketches, normalized formats, and embeddings are new. The cost of profiling is linear in the size of the data, and no later step reads raw data again. In online mode, the profiler runs only on the columns of the arriving table.

## 3.2 Ring Builder

The Ring Builder turns profiles into rings in four steps, shown in the middle of Figure 1. First, it selects *key candidates*: columns with uniqueness $u(c) \geq \tau_u$ and few nulls, which can own a key domain, and small column combinations that are unique together, which can own a composite key. Second, it generates *candidate references* for every column with two indexes over the key candidates: a containment index on the value sketches [38], and an approximate nearest-neighbor index on the embeddings. The first index finds references with overlapping values, and the second finds references whose values differ in format but whose names and descriptions match. Third, it *scopes* and *prunes* the candidates. The scope is one business, and within it the concept tags of the existing pipeline rank candidates, but they do not restrict them: a reference may cross concepts and source systems, because a customer or project key does. Pruning removes candidates that cannot be keys: columns whose type is incompatible with the owner, and generic columns, such as `DATA` or small status codes, that match a large fraction of all key candidates. Fourth, it *forms rings*. Each key candidate with at least one remaining reference starts a ring, and each referencing column joins the ring of its best-scoring owner. When two owners reference each other, such as a master table and its staging copy, the Ring Builder merges their rings and elects the owner with the larger domain. The output is a set of candidate rings, each with an owner, its members, and a score for each member. Section 4 describes these steps in detail.

## 3.3 Budgeted Verifier

The Budgeted Verifier turns candidate rings into confirmed rings, as shown on the right of Figure 1. First, *calibration* draws a random sample of candidates and asks the LLM about them. From these answers, the *cost-based optimizer* sets two thresholds on the score of the distilled model, so that the result keeps a target recall $\gamma$ of the oracle with probability at least $1 - \delta$, in the style of SUPG and BARGAIN [11, 35]. It also chooses how many candidates to place in one prompt. Second, the *distilled model* scores every remaining candidate from its profile features. It accepts candidates above the upper threshold and rejects candidates below the lower threshold. Third, the *ring prompt* verifies each ring that still has uncertain members. One prompt contains the owner, the accepted members, and the uncertain candidates of a ring, and the LLM confirms the owner and returns the members that refer to it. Every LLM answer becomes a label, so the distilled model improves while the verifier runs, and it is stored in a *verdict cache* keyed by the owner and the member. The cache matters most in online mode: when an arrival changes a ring, the verifier asks the LLM only about members it has no verdict for. Fourth, the *SQL join check* of the existing pipeline measures the exact containment of each accepted member in its owner on the warehouse, so that a join accepted from sketches alone is confirmed on the full data. Fifth, *transitive propagation* derives the remaining joins: every confirmed member joins its owner, and two members of the same ring join through it. The output is the join graph $G(\mathcal{R})$ of Section 2.5, which the production pipeline stores in a graph database. Joins that fail downstream, for example a join that multiplies the rows of a master table, return to the verifier as negative labels. Section 5 describes the optimizer and its guarantee.

## 3.4 Ring Maintainer

The Ring Maintainer keeps the rings correct as tables arrive. For each key-like column of the arriving table, the index lookup returns one of three outcomes. First, the column *joins* an existing ring as a member, and the maintainer adds one edge to the ring's owner. Second, the column matches no ring but is a key candidate, and it *starts* a new ring that later arrivals can join. Third, the column *challenges* the owner of a ring: it is unique, and the current owner and its members are contained in it, as when the true customer master arrives after many tables that reference customers. The maintainer then elects the new owner and re-points the edges of that ring, which changes $|M_R|$ edges and no other ring. An arrival can also connect two rings, for example a staging copy that overlaps

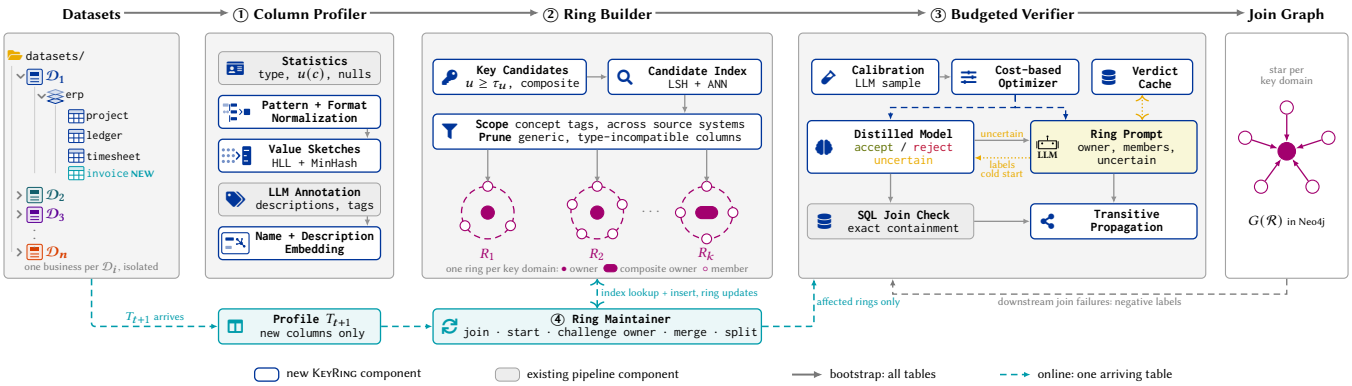



**Figure 1: KeyRing architecture: the Column Profiler summarizes every column in one pass without the LLM; the Ring Builder groups candidate key columns into rings, one per key domain, each with an owner; the Budgeted Verifier accepts or rejects confident candidates with a distilled model, verifies each ring with one batched LLM prompt, and propagates joins by transitivity, so that the join graph keeps a target recall of the all-pairs LLM baseline. In online mode (dashed teal), an arriving table $T_{t+1}$ is profiled alone, and the Ring Maintainer looks its columns up in the candidate index, updates the affected rings, and sends only those rings to the verifier. Each business $\mathcal{D}_i$ is processed in isolation.**

both a master and its older export, and the maintainer merges them. When the verifier rejects part of a ring, the maintainer splits it. Every change touches only the rings the new columns map to, so the cost of an arrival is independent of the number of tables. The verifier treats each changed ring as uncertain and checks it with one ring prompt, together with the uncertain candidates of the new table. Section 6 gives the update rules and shows that the final rings do not depend on the order of arrival.

*Example 3.1 (Running Example).* In the services firm of Section 1, the profiler computes the profile of the project master's PROJECT_CODE, which is unique, and of the $k$ columns that reference it. Normalization removes the leading zeros of the padded export, so its codes are contained in the master again. The containment index proposes PRJ_ID and the padded column as references, and the embedding index proposes ENGAGEMENT, whose values carry a prefix. The Ring Builder prunes DATA, which matches [X]% of all key candidates, and merges the staging copy of the master into the same ring. The distilled model accepts the confident members and rejects the surrogate ID columns, whose values are contained in the codes by accident. The LLM then verifies the ring with a single prompt that lists the uncertain members, instead of answering $\binom{k+1}{2}$ pairwise prompts.

## 4 Column Profiling and Ring Building

**[Key candidate selection, the two candidate indexes, pruning of generic columns, ring formation and owner election, complexity.]**

## 5 Budgeted Verification

**[Distilled model, threshold selection with recall guarantee, ring prompts and batching, transitive propagation, cost analysis.]**

## 6 Online Ring Maintenance

**[Arrival algorithm (profile, index lookup, join/start/challenge), owner re-election, ring merge and split, keeping the recall guarantee after every arrival, order independence (bootstrap = one arrival of all tables), cost per arrival.]**

## 7 Experiments

**[Datasets, baselines (non-LLM, all-pairs LLM, hybrid), cost and quality results.] [Online experiments: replay each business's tables in creation order; LLM calls, tokens, and latency per arrival vs. pairwise prompting (new table × all tables) and a bootstrap rerun; recall after each arrival; graph after the last arrival vs. a single bootstrap run; random arrival orders for order independence.] [Redundant edges downstream: number of distinct shortest join paths per table pair, how often a shortcut edge changes the chosen path, and prompt tokens of the full vs. the ring graph.]**

## 8 Conclusion

**[Conclusion.]**

## References


[1] Ziawasch Abedjan, Lukasz Golab, and Felix Naumann. 2015. Profiling Relational Data: A Survey. *VLDB J.* 24, 4 (2015), 557–581.

[2] Fabian Biester, Mohamed Abdelaal, and Daniel Del Gaudio. 2024. LLM-Clean: Context-Aware Tabular Data Cleaning via LLM-Generated OFDs. *CoRR* abs/2404.18681 (2024).

[3] Ting Cai, Rakesh R. Menon, Yiru Chen, Zifan Liu, Yuan Tian, Fei Wu, Anudeep Chimakurthi, Prashanthi Ramamurthy, Sunav Choudhary, Kun Qian, and Yunyao Li. 2026. TACO: Task-Aware Column Description Generation Using LLMs. *CoRR* abs/2606.21685 (2026).

[4] Lingjiao Chen, Matei Zaharia, and James Zou. 2023. FrugalGPT: How to Use Large Language Models While Reducing Cost and Improving Performance. *CoRR* abs/2305.05176 (2023).

[5] Yuhao Deng, Chengliang Chai, Lei Cao, Qin Yuan, Siyuan Chen, Yanrui Yu, Zhaoze Sun, Junyi Wang, Jiajun Li, Ziqi Cao, Kaisen Jin, Chi Zhang, Yuqing Jiang, Yuanfang Zhang, Yuping Wang, Ye Yuan, Guoren Wang, and Nan Tang. 2024. LakeBench: A Benchmark for Discovering Joinable and Unionable Tables in Data Lakes. *Proc. VLDB Endow.* 17, 8 (2024), 1925–1938.

[6] Zhihao Ding, Yongkang Sun, and Jieming Shi. 2026. Retrieve-and-Verify: A Table Context Selection Framework for Accurate Column Annotations. In *SIGMOD*. doi:10.1145/3769823

[7] Yuyang Dong, Chuan Xiao, Takuma Nozawa, Masafumi Enomoto, and Masafumi Oyamada. 2023. DeepJoin: Joinable Table Discovery with Pre-trained Language Models. *Proc. VLDB Endow.* 16, 10 (2023), 2458 – 2470. doi:10.14778/3603581.3603587

[8] Mahdi Esmailoghli, Jorge-Arnulfo Quiané-Ruiz, and Ziawasch Abedjan. 2022. MATE: Multi-Attribute Table Extraction. *Proc. VLDB Endow.* 15, 8 (2022).

[9] Benjamin Feuer, Yurong Liu, Chinmay Hegde, and Juliana Freire. 2024. ArcheType: A Novel Framework for Open-Source Column Type Annotation using Large Language Models. *Proc. VLDB Endow.* 17, 9 (2024), 2279–2292. doi:10.14778/3665844.3665857

[10] Lan Jiang and Felix Naumann. 2020. Holistic Primary Key and Foreign Key Detection. *J. Intell. Inf. Syst.* 54, 3 (2020).

[11] Daniel Kang, Edward Gan, Peter Bailis, Tatsunori Hashimoto, and Matei Zaharia. 2020. Approximate Selection with Guarantees using Proxies. *Proc. VLDB Endow.* 13, 11 (2020).

[12] Keti Korini and Christian Bizer. 2023. Column Type Annotation using ChatGPT. In *Joint Proceedings of Workshops at the 49th International Conference on Very Large Data Bases (VLDB 2023), Vancouver, Canada, August 28 - September 1, 2023 (CEUR Workshop Proceedings, Vol. 3462)*, Rajesh Bordawekar, Cinzia Cappiello, Vasilis Efthymiou, Lisa Ehrlinger, Vijay Gadepally, Sainyam Galhotra, Sandra Geisler, Sven Groppe, Le Gruenwald, Alon Y. Halevy, Hazar Harmouch, Oktie Hassanzadeh, Ihab F. Ilyas, Ernesto Jiménez-Ruiz, Sanjay Krishnan, Tirthankar Lahiri, Guoliang Li, Jiaheng Lu, Wolfgang Mauerer, Umar Farooq Minhas, Felix Naumann, M. Tamer Özsu, El Kindi Rezig, Kavitha Srinivas, Michael Stonebraker, Satyanarayana R. Valluri, Maria-Esther Vidal, Haixun Wang, Jiannan Wang, Yingjun Wu, Xun Xue, Mohamed Zaït, and Kai Zeng (Eds.). CEUR-WS.org. https://ceur-ws.org/Vol-3462/TADA1.pdf

[13] Sebastian Kruse, David Hahn, Marius Walter, and Felix Naumann. 2017. Metacrate: Organize and Analyze Millions of Data Profiles. In *CIKM*. 2483–2486. doi:10.1145/3132847.3133180

[14] Sebastian Kruse and Felix Naumann. 2018. Efficient Discovery of Approximate Dependencies. *Proc. VLDB Endow.* 11, 7 (2018), 759–772.

[15] Sebastian Kruse, Thorsten Papenbrock, Christian Dullweber, Moritz Finke, Manuel Hegner, Martin Zabel, Christian Zöllner, and Felix Naumann. 2017. Fast Approximate Discovery of Inclusion Dependencies. In *BTW*. 207–226.

[16] Lukas Laskowski, Fabian Panse, Michael Hladik, Jan Portisch, and Felix Naumann. 2025. Schuyler: Self-Supervised Clustering of Tables in Relational Databases. *Proc. VLDB Endow.* 19, 4 (2025), 657–669.

[17] Fangyu Lei et al. 2025. Spider 2.0: Evaluating Language Models on Real-World Enterprise Text-to-SQL Workflows. In *ICLR*.

[18] Yiming Lin, Yeye He, and Surajit Chaudhuri. 2023. Auto-BI: Automatically Build BI-Models Leveraging Local Join Prediction and Global Schema Graph Optimization. *Proc. VLDB Endow.* 16, 10 (2023).

[19] Chunwei Liu et al. 2025. Palimpzest: Optimizing AI-Powered Analytics with Declarative Query Processing. In *CIDR*.

[20] Yurong Liu, Eduardo H. M. Pena, Aécio S. R. Santos, Eden Wu, and Juliana Freire. 2025. Magneto: Combining Small and Large Language Models for Schema Matching. *Proc. VLDB Endow.* 18 (2025).

[21] Marc Maynou et al. 2024. Freyja: Efficient Join Discovery in Data Lakes. *CoRR* abs/2412.06637 (2024).

[22] Amith Nagarajan and Thomas Altman. 2026. DBAutoDoc: Automated Discovery and Documentation of Undocumented Database Schemas via Statistical Analysis and Iterative LLM Refinement. *CoRR* abs/2603.23050 (2026).

[23] Avanika Narayan, Ines Chami, Laurel Orr, and Christopher Ré. 2022. Can Foundation Models Wrangle Your Data? *Proc. VLDB Endow.* 16, 4 (dec 2022), 738–746. doi:10.14778/3574245.3574258

[24] Thorsten Papenbrock, Tanja Bergmann, Moritz Finke, Jakob Zwiener, and Felix Naumann. 2015. Data Profiling with Metanome. *Proc. VLDB Endow.* 8, 12 (2015), 1860–1863. doi:10.14778/2824032.2824086

[25] Thorsten Papenbrock, Sebastian Kruse, Jorge-Arnulfo Quiané-Ruiz, and Felix Naumann. 2015. Divide & Conquer-based Inclusion Dependency Discovery. *Proc. VLDB Endow.* 8, 7 (2015), 774–785.

[26] Thorsten Papenbrock and Felix Naumann. 2016. A Hybrid Approach to Functional Dependency Discovery. In *SIGMOD*. 821–833. doi:10.1145/2882903.2915203

[27] Thorsten Papenbrock and Felix Naumann. 2017. A Hybrid Approach for Efficient Unique Column Combination Discovery. In *BTW*. 195–204.

[28] Liana Patel, Siddharth Jha, Carlos Guestrin, and Matei Zaharia. 2024. LOTUS: Enabling Semantic Queries with LLMs Over Tables of Unstructured and Structured Data. *CoRR* abs/2407.11418 (2024).

[29] Alexandra Rostin, Oliver Albrecht, Jana Bauckmann, Felix Naumann, and Ulf Leser. 2009. A Machine Learning Approach to Foreign Key Discovery. In *WebDB*.

[30] Philipp Schirmer, Thorsten Papenbrock, Sebastian Kruse, Felix Naumann, Dennis Hempfing, Torben Mayer, and Daniel Neuschäfer-Rube. 2019. DynFD: Functional Dependency Discovery in Dynamic Datasets. In *EDBT*. 253–264.

[31] Vladislav Shkapenyuk, Divesh Srivastava, Theodore Johnson, and Parisa Ghane. 2025. Automatic Metadata Extraction for Text-to-SQL. *CoRR* abs/2505.19988 (2025).

[32] Fabian Tschirschnitz, Thorsten Papenbrock, and Felix Naumann. 2017. Detecting Inclusion Dependencies on Very Many Tables. *ACM Trans. Database Syst.* 42, 3 (2017).

[33] Xiaolong Wan and Xixian Han. 2026. Not Every Dependency Is Worth Discovering: Toward Value-Driven Data Dependency Discovery. *CoRR* abs/2607.22219 (2026).

[34] Sepanta Zeighami, Shreya Shankar, and Aditya Parameswaran. 2025. Featurized-Decomposition Join: Low-Cost Semantic Joins with Guarantees. *CoRR* abs/2512.05399 (2025).

[35] Sepanta Zeighami, Shreya Shankar, and Aditya Parameswaran. 2026. Cut Costs, Not Accuracy: LLM-Powered Data Processing with Guarantees. In *SIGMOD*.

[36] Meihui Zhang, Marios Hadjieleftheriou, Beng Chin Ooi, Cecilia M. Procopiuc, and Divesh Srivastava. 2010. On Multi-Column Foreign Key Discovery. *Proc. VLDB Endow.* 3, 1 (2010).

[37] Erkang Zhu, Dong Deng, Fatemeh Nargesian, and Renée J. Miller. 2019. JOSIE: Overlap Set Similarity Search for Finding Joinable Tables in Data Lakes. In *Proceedings of the 2019 International Conference on Management of Data, SIGMOD Conference 2019, Amsterdam, The Netherlands, June 30 - July 5, 2019*, Peter A. Boncz, Stefan Manegold, Anastasia Ailamaki, Amol Deshpande, and Tim Kraska (Eds.). ACM, 847–864. doi:10.1145/3299869.3300065

[38] Erkang Zhu, Fatemeh Nargesian, Ken Q. Pu, and Renée J. Miller. 2016. LSH Ensemble: Internet-Scale Domain Search. *Proc. VLDB Endow.* 9, 12 (2016), 1185–1196. doi:10.14778/2994509.2994534